

ANS	Projekt	Akademia Nauk Stosowanych w Nowym Sączu	
	Programowanie współbieżne i rozproszone	Wydział Nauk Inżynieryjnych	
Projekt nr: P2	Temat: Wątki POSIX — biblioteka Pthreads	Data wykonania: 20.03.2026	Data oddania: 21.03.2026
		Prowadzący: mgr inż. Daniel Drozd	
Wykonał: Henryk Szambelan		Ocena:	
Numer albumu: 33839			
Kierunek: INS	Grupa projektowa: P NST		

I. Cel ćwiczenia

Celem niniejszych zajęć jest konfiguracja środowiska programistycznego obsługującego standard **POSIX Threads (Pthreads)** na systemie Windows oraz opanowanie podstawowych funkcji biblioteki służących do tworzenia i zarządzania wątkami.

Efekty kształcenia

Po ukończeniu zajęć student powinien:

- mieć poprawnie skonfigurowane środowisko MSYS2 + MinGW-w64 + VS Code,
- rozumieć różnicę między procesem a wątkiem,
- tworzyć wątki za pomocą funkcji `pthread_create()`,
- czekać na zakończenie wątków przez `pthread_join()`,
- przekazywać parametry do wątku i odbierać wyniki,
- poprawnie kompilować programy z flagą `-lpthread`.

II. Zadania

Zadanie 1

1. Wersja kompilatora GCC

Poniżej przedstawiono zrzut ekranu z terminala weryfikujący zainstalowaną wersję kompilatora GCC w środowisku.

```
PS C:\Users\henry\Desktop\P2> gcc --version
gcc.exe (Rev13, Built by MSYS2 project) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Rysunek 1: Wynik wywołania polecenia `gcc --version`

2. Kod źródłowy zmodyfikowanego programu

Zmodyfikowany program tworzy wątek poboczny, który trzykrotnie wypisuje swój numer, stosując opóźnienie 300 ms (przy użyciu funkcji `usleep(300000)` z biblioteki `unistd.h`).

```
1  #include <pthread.h>
2  #include <stdio.h>
3  #include <unistd.h>
4
5  void* funkcja_watku(void* arg) {
6      int numer = *(int*)arg;
7
8      for (int i = 0; i < 3; i++) {
9          printf("Witaj z watku numer %d!\n", numer);
10         usleep(300000);
11     }
12
13     return NULL;
14 }
15
16 int main(void) {
17     pthread_t tid;
18     int param = 1;
19
20     printf("Główny wątek startuje.\n");
21
22     int ret = pthread_create(&tid, NULL, funkcja_watku, &param);
23     if (ret != 0) {
24         printf("Błąd tworzenia watku: %d\n", ret);
25         return 1;
26     }
27
28     // Zakomentowano w celu testów w drugiej części zadania
29     // pthread_join(tid, NULL);
30
31     printf("Główny wątek kończy pracę. Koniec programu.\n");
32
33     return 0;
34 }
```

Listing 1: Kod programu z wątkami POSIX

3. Output z działania programu

Wersja z `pthread_join`:

```

● PS C:\Users\henry\Desktop\P2> ./Z1.exe
Główny wątek startuje.
Witaj z watku numer 1!
Witaj z watku numer 1!
Witaj z watku numer 1!
Wątek zakończył prace. Koniec programu.
○ PS C:\Users\henry\Desktop\P2>

```

Rysunek 2: Wynik działania programu z aktywną funkcją `pthread_join`

Wersja bez `pthread_join`:

```

● PS C:\Users\henry\Desktop\P2> ./Z1.exe
Główny wątek startuje.
Główny wątek kończy prace. Koniec programu.
○ PS C:\Users\henry\Desktop\P2>

```

Rysunek 3: Wynik działania programu po usunięciu funkcji `pthread_join`

4. Wnioski po usunięciu `pthread_join`

Po usunięciu z kodu funkcji `pthread_join`, wątek główny nie zatrzymuje się, aby poczekać na zakończenie działania wątku pobocznego. W rezultacie program natychmiastowo dochodzi do instrukcji `return 0`; i proces kończy się, zanim wątek poboczny zdąży wykonać swoją pętlę i wypisać komunikaty. Dzieje się tak, ponieważ zakończenie działania całego procesu przez system operacyjny zmusza do natychmiastowego zamknięcia wszystkich należących do niego wątków, niezależnie od stanu ich wykonania.

Zadanie 2

1. Kod źródłowy programu z 6 wątkami i obliczaniem sumy

Poniżej znajduje się kod programu zmodyfikowany tak, aby uruchamiał 6 wątków. Każdy z wątków oblicza sumę liczb od 1 do n , gdzie n jest równe numerowi wątku pomnożonemu przez 1000.

```

1  #include <pthread.h>
2  #include <stdio.h>
3  #include <unistd.h>
4
5  #define LICZBA_WATKOW 6 // Zmiana liczby watkow na 6
6
7  void* pracownik(void* arg) {
8      int id = *(int*)arg;
9      printf("[Wątek %d] Rozpoczynam prace...\n", id);
10
11     // Obliczanie n (numer watku * 1000)
12     int n = id * 1000;

```

```

13     long long suma = 0;
14
15     // Obliczanie sumy od 1 do n
16     for (int i = 1; i <= n; i++) {
17         suma += i;
18     }
19
20     usleep(500000 * id);    // Opoznienie: mikrosekundy (500ms * id)
21
22     // Wypisanie wyniku wewnatrz funkcji watku
23     printf("[Watek %d] Praca zakonczona! Suma od 1 do %d wynosi: %lld\n",
24            id, n, suma);
25
26     return NULL;
27 }
28
29 int main(void) {
30     pthread_t tids[LICZBA_WATKOW];
31     int parametry[LICZBA_WATKOW];
32
33     for (int i = 0; i < LICZBA_WATKOW; i++) {
34         parametry[i] = i + 1;
35         int ret = pthread_create(&tids[i], NULL, pracownik, &parametry[i]);
36         if (ret != 0) {
37             printf("Blad tworzenia watku %d: %d\n", i + 1, ret);
38             return 1;
39         }
40     }
41
42     printf("[Main] Czekam na zakonczenie watkow...\n");
43
44     for (int i = 0; i < LICZBA_WATKOW; i++) {
45         pthread_join(tids[i], NULL);
46     }
47
48     printf("[Main] Wszystko gotowe!\n");
49     return 0;
50 }

```

Listing 2: Wielowatkowe obliczanie sumy

2. Output z działania programu

Poniżej przedstawiono zrzuty ekranu z dwóch niezależnych uruchomień programu, które pozwalają zaobserwować kolejność wykonywania poszczególnych wątków.

```

PS C:\Users\henry\Desktop\P2> ./Z2.exe
[Main] Czekam na zakończenie watkow...
[Watek 4] Rozpoczynam prace...
[Watek 2] Rozpoczynam prace...
[Watek 1] Rozpoczynam prace...
[Watek 5] Rozpoczynam prace...
[Watek 6] Rozpoczynam prace...
[Watek 3] Rozpoczynam prace...
[Watek 1] Praca zakończona! Suma od 1 do 1000 wynosi: 500500
[Watek 2] Praca zakończona! Suma od 1 do 2000 wynosi: 2001000
[Watek 3] Praca zakończona! Suma od 1 do 3000 wynosi: 4501500
[Watek 4] Praca zakończona! Suma od 1 do 4000 wynosi: 8002000
[Watek 5] Praca zakończona! Suma od 1 do 5000 wynosi: 12502500
[Watek 6] Praca zakończona! Suma od 1 do 6000 wynosi: 18003000
[Main] Wszystko gotowe!
PS C:\Users\henry\Desktop\P2> ./Z2.exe

```

Rysunek 4: Wynik z pierwszego uruchomienia programu (Zadanie 2)

```

PS C:\Users\henry\Desktop\P2> ./Z2.exe
[Main] Czekam na zakończenie watkow...
[Watek 1] Rozpoczynam prace...
[Watek 4] Rozpoczynam prace...
[Watek 5] Rozpoczynam prace...
[Watek 3] Rozpoczynam prace...
[Watek 6] Rozpoczynam prace...
[Watek 2] Rozpoczynam prace...
[Watek 1] Praca zakończona! Suma od 1 do 1000 wynosi: 500500
[Watek 2] Praca zakończona! Suma od 1 do 2000 wynosi: 2001000
[Watek 3] Praca zakończona! Suma od 1 do 3000 wynosi: 4501500
[Watek 4] Praca zakończona! Suma od 1 do 4000 wynosi: 8002000
[Watek 5] Praca zakończona! Suma od 1 do 5000 wynosi: 12502500
[Watek 6] Praca zakończona! Suma od 1 do 6000 wynosi: 18003000
[Main] Wszystko gotowe!
PS C:\Users\henry\Desktop\P2>

```

Rysunek 5: Wynik z drugiego uruchomienia programu (Zadanie 2)

3. Analiza kolejności komunikatów

Nie, kolejność komunikatów na początku działania wątków nie jest ściśle deterministyczna. Wynika to z faktu, że o przydziale czasu procesora decyduje planista systemu operacyjnego (scheduler), przez co komunikaty o rozpoczęciu pracy ("Rozpoczynam prace...") mogą przeplatać się w różnej kolejności. Z kolei komunikaty o zakończeniu pracy pojawiają się w określonym porządku (od wątku 1 do 6), ale jest to wymuszone wyłącznie przez sztucznie wprowadzone, rosnące opóźnienia (`usleep(500000 * id)`), a nie przez naturalny determinizm środowiska wielowątkowego.

Zadanie 3

1. Kod źródłowy programu ze strukturą i wątkami

Poniżej znajduje się kod programu wykorzystujący strukturę DaneWatku do przekazania złożonych argumentów do wątków oraz odebrania od nich wyników (obliczonych silni).

```
1  #include <pthread.h>
2  #include <stdio.h>
3
4  #define LICZBA_WATKOW 4
5
6  // 1. Definicja struktury DaneWatku
7  typedef struct {
8      int id;           // Numer watku
9      int n;           // Liczba do obliczenia silni
10     long long wynik;  // Miejsce na wynik
11 } DaneWatku;
12
13 // Funkcja wykonywana przez kazdy watek
14 void* oblicz_silnie(void* arg) {
15     // Rzutowanie wskaźnika na typ naszej struktury
16     DaneWatku* dane = (DaneWatku*)arg;
17
18     long long silnia = 1;
19
20     // Obliczanie n!
21     for (int i = 1; i <= dane->n; i++) {
22         silnia *= i;
23     }
24
25     // Zapisanie wyniku do pola w strukturze
26     dane->wynik = silnia;
27
28     return NULL;
29 }
30
31 int main(void) {
32     pthread_t tids[LICZBA_WATKOW];
33     DaneWatku dane_watkow[LICZBA_WATKOW];
34
35     // Przykładowe wartosci n do policzenia
36     int wartosci_n[LICZBA_WATKOW] = {5, 8, 10, 12};
37
38     // 2. Tworzenie 4 watkow
39     for (int i = 0; i < LICZBA_WATKOW; i++) {
40         dane_watkow[i].id = i + 1;
41         dane_watkow[i].n = wartosci_n[i];
42         dane_watkow[i].wynik = 0; // Inicjalizacja zerem
43
44         int ret = pthread_create(&tids[i], NULL, oblicz_silnie, &
45                                 dane_watkow[i]);
46         if (ret != 0) {
47             printf("Bład tworzenia watku %d: %d\n", i + 1, ret);
48             return 1;
49         }
50     }
```

```

51 // Czekamy na zakończenie pracy wszystkich wątków
52 for (int i = 0; i < LICZBA_WATKOW; i++) {
53     pthread_join(tids[i], NULL);
54 }
55
56 // 3. Wypisanie wyników w głównym wątku
57 printf("--- Wyniki obliczeń z wątków ---\n");
58 for (int i = 0; i < LICZBA_WATKOW; i++) {
59     printf("Wątek %d obliczył %d! = %lld\n", dane_watkow[i].id,
60           dane_watkow[i].n, dane_watkow[i].wynik);
61 }
62 return 0;
63 }

```

Listing 3: Przekazywanie złożonych danych do wątków

2. Output z działania programu

Poniżej przedstawiono zrzut ekranu prezentujący poprawne obliczenie wartości silni przez poszczególne wątki.

```

PS C:\Users\henry\Desktop\P2> ./Z3.exe
--- Wyniki obliczeń z wątków ---
Wątek 1 obliczył 5! = 120
Wątek 2 obliczył 8! = 40320
Wątek 3 obliczył 10! = 3628800
Wątek 4 obliczył 12! = 479001600
PS C:\Users\henry\Desktop\P2>

```

Rysunek 6: Wynik działania programu obliczającego silnię w wątkach (Zadanie 3)

3. Analiza sposobu odczytywania wyników

Wyniki z wątków odczytujemy dopiero po wywołaniu funkcji `pthread_join`, ponieważ gwarantuje ona, że dany wątek w pełni zakończył już swoje działanie. Gdybyśmy próbowali odczytać pole `wynik` wcześniej, istniałoby wysokie ryzyko odczytania zera lub danych niekompletnych (zanim wątek zdążyłby je w ogóle obliczyć i zapisać w pamięci). Użycie `pthread_join` działa zatem jako punkt synchronizacyjny, upewniając wątek główny, że struktury danych są gotowe i bezpieczne do wyświetlenia.

III. Pytania kontrolne

1. Czym różni się proces od wątku? Podaj przynajmniej trzy różnice.

Proces to niezależny program posiadający własną, chronioną przestrzeń adresową, podczas gdy wątek to ścieżka wykonania wewnątrz procesu, która dzieli pamięć z innymi wątkami należącymi do tego samego procesu. Główne różnice to: 1) procesy posiadają oddzielną przestrzeń adresową, a wątki współdzielą pamięć (zmienne globalne, stertę), 2) przełączanie kontekstu (context switch) między wątkami jest znacznie szybsze i mniej obciąża system niż przełączanie między procesami, 3) błąd krytyczny w jednym wątku (np.

naruszenie ochrony pamięci) zazwyczaj "zabija" cały proces, podczas gdy awaria jednego procesu nie ma wpływu na stabilność innych procesów w systemie.

2. Co to jest standard POSIX i dlaczego korzystamy z biblioteki Pthreads zamiast natywnego Windows API?

POSIX (Portable Operating System Interface) to rodzina standardów określona przez IEEE, która ma na celu zapewnienie kompatybilności i przenośności oprogramowania pomiędzy różnymi systemami operacyjnymi. Korzystamy z biblioteki Pthreads, ponieważ pozwala ona na tworzenie przenośnego kodu. Dzięki niej, ten sam program z wątkami może zostać bez żadnych modyfikacji skompilowany i uruchomiony na systemach Linux, macOS, a także w środowisku Windows (przy użyciu zestawu narzędzi MinGW-w64 lub MSYS2), co byłoby niemożliwe przy bezpośrednim użyciu funkcji z Windows API.

3. Co zwraca `pthread_create()` w przypadku błędu? Czym różni się to od konwencji większości funkcji w C?

Funkcja `pthread_create()` w przypadku błędu zwraca bezpośrednio kod błędu (czyli dodatnią liczbę całkowitą, np. `EAGAIN` lub `EINVAL`), a w przypadku sukcesu zwraca 0. Stanowi to wyjątek od klasycznej konwencji bibliotek języka C, w której funkcje zazwyczaj zwracają -1 (lub `NULL`) w razie niepowodzenia i ustawiają globalną zmienną `errno`. Pthreads unika modyfikacji globalnej zmiennej `errno`, aby zapewnić bezpieczeństwo wielowątkowe (tzw. thread-safety) i uniknąć sytuacji, w której wiele wątków próbowałoby jednocześnie nadpisać ten sam kod błędu.

4. Dlaczego każdy wątek powinien zostać dołączony (`pthread_join`) lub odłączony (`pthread_detach`)? Co to jest zombie thread?

Każdy uruchomiony wątek rezerwuje określone zasoby systemowe, takie jak chociażby pamięć na własny stos oraz deskryptor wątku. Dołączenie wątku (`pthread_join`) lub jego odłączenie (`pthread_detach`) informuje system operacyjny, że te zasoby mogą zostać bezpiecznie zwolnione po jego zakończeniu. Jeśli wątek zakończy swoje działanie, ale programista nie wywoła żadnej z tych funkcji, staje się on tzw. wątkiem zombie (zombie thread) — bezużytecznym tworem, który marnuje zasoby, co w programach tworzących masowo nowe wątki może doprowadzić do wycieku pamięci.

5. Dlaczego nie można zwracać wskaźnika do lokalnej zmiennej z funkcji wątku? Jak prawidłowo przekazać wynik z wątku do wątku głównego?

Lokalne zmienne w funkcji wątku są tworzone na jego indywidualnym stosie, który ulega zniszczeniu zaraz po tym, jak wątek zakończy swoje działanie (wykona instrukcję `return`). Zwrócenie wskaźnika do takiej zmiennej wywoła niezdefiniowane zachowanie (dangling pointer), ponieważ wskaźnik będzie wskazywał na zwolniony już obszar pamięci. Aby prawidłowo zwrócić wynik, należy użyć pamięci alokowanej dynamicznie (np. za pomocą `malloc`), zapisać wynik w zmiennej globalnej lub przekazać wskaźnik do struktury przygotowanej wcześniej w wątku głównym (tak jak w rozwiązaniu zadania 3).

6. Co się dzieje z wątkami potomnymi, gdy wątek główny (`main`) zakończy działanie?

Gdy wątek główny (wykonujący funkcję `main`) kończy działanie poprzez instrukcję `return` lub funkcję `exit()`, proces ulega natychmiastowemu zamknięciu przez system operacyjny. Skutkuje to bezwarunkowym i gwałtownym przerwaniem pracy ("zabiciem") wszystkich innych wątków potomnych należących do tego procesu. Z tego względu niezwykle istotne jest stosowanie `pthread_join`, które wstrzymuje zakończenie głównego wątku do momentu, aż wszystkie pozostałe zakończą swoje zadania.

7. **Czym jest niedeterminizm w programach wielowątkowych? Podaj przykład z zajęć gdzie mógł wystąpić.**

Niedeterminizm w programowaniu współbieżnym to zjawisko polegające na tym, że dokładna kolejność wykonywania poszczególnych instrukcji przez różne wątki jest niemożliwa do przewidzenia, ponieważ decyduje o tym wewnętrzny planista (scheduler) systemu operacyjnego. Przykładem z przeprowadzonych ćwiczeń (Zadanie 2) była kolejność wypisywania na ekranie komunikatu "Rozpaczynam prace..." przez 6 wątków. Mimo że były one powoływane w pętli od 1 do 6, komunikaty te przeplatały się w losowej i różnej przy każdym uruchomieniu programu kolejności.

8. **Do czego służy flaga `-lpthread` podczas kompilacji? Co się stanie jeśli ją pominiemy?**

Flaga `-lpthread` przekazywana kompilatorowi `gcc` (a dokładniej linkerowi) stanowi instrukcję polecającą mu konsolidację programu z właściwą biblioteką POSIX Threads (`libpthread`). Jeśli pominiemy tę flagę w procesie budowania, faza kompilacji co prawda się powiedzie, ale faza linkowania zakończy się błędami typu "undefined reference". Oznacza to, że kompilator nie będzie wiedział, gdzie fizycznie znajduje się kod wykonywalny dla funkcji takich jak `pthread_create` czy `pthread_join`, przez co nie wygeneruje pliku `.exe`.